



# ECA304

## PROGRAMAÇÃO

### ORIENTADA A OBJETOS

Alexandre H. MOREIRA

CLASSES E OBJETOS - HERANÇA



# O que é Herança?

Herança é quando uma classe filha reutiliza código de uma classe base, podendo especializá-la.

- Classe base: define atributos e métodos genéricos.
- Subclasse: herda e pode sobrescrever métodos da classe base.
- Usamos quando há uma relação do tipo: "é um(a)"
  - Um **Cachorro** é um **Animal**
  - Um **CarroEsportivo** é um **Carro**



# Terminologia importante

| Termo       | Significado                                     |
|-------------|-------------------------------------------------|
| Superclasse | Classe que será herdada (ex: Animal)            |
| Subclasse   | Classe que herda (ex: Cachorro)                 |
| Herança     | Mecanismo de compartilhamento de código         |
| super()     | Permite acessar métodos da superclasse          |
| Sobrescrita | Redefinir um método da superclasse na subclasse |



# Vamos discutir a Herança



## Exemplo sem herança

```
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def falar(self):
        print("O animal emite um som.")

class Cachorro():
    def __init__(self, nome, cor):
        self.nome = nome
        self.cor = cor

    def falar(self):
        print(f"{self.nome} (cor {self.cor}) diz: Auau.")
```



# Vamos discutir a Herança



## Exemplo com herança

```
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def falar(self):
        print("O animal emite um som.")

class Cachorro(Animal):
    pass
```



# Vamos discutir a Herança



## Exemplo com **super()**

```
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def falar(self):
        print("O animal emite um som.")

class Cachorro(Animal):
    def __init__(self, nome, cor):
        super().__init__(nome) # chama o __init__ da superclasse
        self.cor = cor

    def falar(self):
        print(f"{self.nome} (cor {self.cor}) diz: Auau.")
```



# Tipos de herança

- **Herança múltipla:** Uma subclasse herda de duas ou mais superclasses.

```
class GPS:
    def localizar(self):
        print("Localizando...")

class ComputadorDeBordo:
    def exibir_dados(self):
        print("Exibindo informações do carro")

class CarroInteligente(GPS, ComputadorDeBordo):
    pass
```

● *Cuidado com conflitos de métodos e ordem de resolução!*



# Tipos de herança

- **Herança simples:** Uma subclasse herda de uma única superclasse.

```
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def falar(self):
        print("O animal emite um som.")

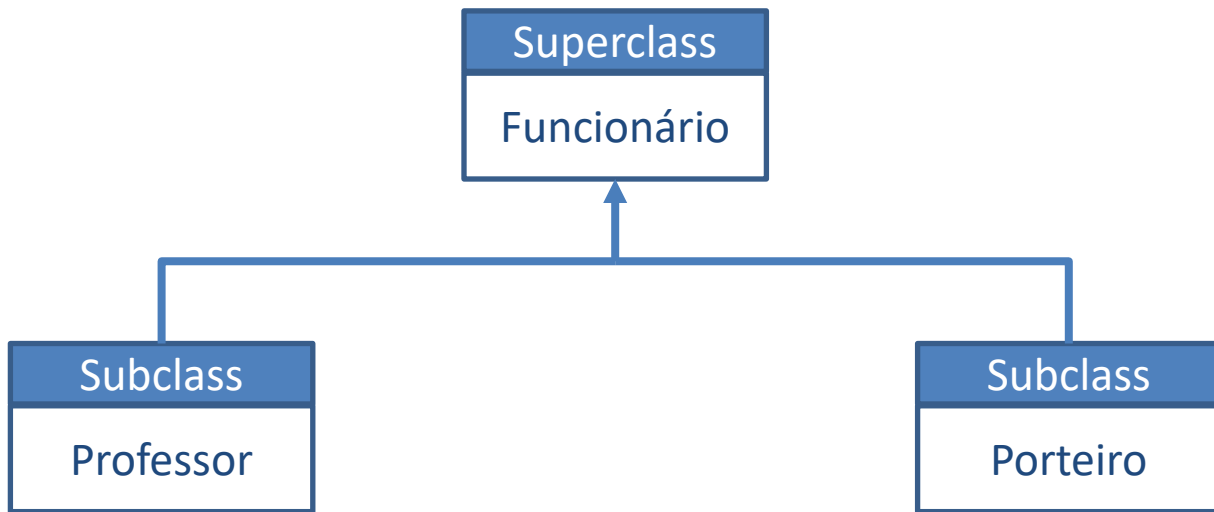
class Cachorro(Animal):
    pass
```





# Tipos de herança

- **Herança hierárquica:** Várias classes herdam de uma mesma superclasse.





# Tipos de herança

- **Herança hierárquica:** Várias classes herdam de uma mesma superclasse.

```
class Funcionario:
    def trabalhar(self):
        print("Trabalhando...")

class Professor(Funcionario):
    def dar_aula(self):
        print("Dando aula")

class Porteiro(Funcionario):
    def abrir_portao(self):
        print("Abrindo portão")
```



# Boas práticas e armadilhas

- Quando **usar** herança:
  - Quando há uma relação "**é um tipo de**";
  - Para evitar duplicação de código;
  - Para construir hierarquias bem definidas.



# Boas práticas e armadilhas

- Quando **evitar** herança:

- Quando a relação não é "é um", mas sim "tem um"

```
class Motor:
    def ligar(self):
        print("Motor ligado.")

class Carro(Motor): # Parece conveniente, mas está errado!
    def dirigir(self):
        print("Carro em movimento.")
```



## Problema:

A herança diz que *um carro é um motor*, mas na verdade um **carro tem um motor**.



# Boas práticas e armadilhas

## □ Correção com **composição**:

```
class Motor:
    def ligar(self):
        print("Motor ligado.")

class Carro:
    def __init__(self):
        self.motor = Motor()

    def dirigir(self):
        self.motor.ligar()
        print("Carro em movimento.")
```

## ● **Vantagem:**

Separação clara das responsabilidades. Cada classe faz **apenas o que lhe compete**, sem herdar indevidamente.



# Boas práticas e armadilhas

- Quando as subclasses não especializam claramente a superclasse:

```
class Animal:
    def comer(self):
        print("Comendo...")

class Impressora(Animal): # Não faz sentido!
    def imprimir(self):
        print("Imprimindo documento...")
```



**Problema:** A classe Impressora não é um tipo de Animal. O código pode até funcionar, mas **a modelagem está conceitualmente errada.**



**Lembre-se:** Herança deve refletir uma relação lógica e semântica entre as classes.



# Boas práticas e armadilhas

- Quando a herança dificulta a manutenção e acoplamento:

```
class Funcionario:
    def calcular_pagamento(self):
        print("Pagamento base")

class Estagiario(Funcionario):
    def calcular_pagamento(self):
        print("Pagamento com desconto")

class Voluntario(Funcionario): # Não recebe pagamento!
    pass
```



**Problema:** Voluntario **não deveria herdar** o comportamento de Funcionario, pois herda métodos que **não fazem sentido para ele**, como `calcular_pagamento()`.



# Boas práticas e armadilhas

- Quando **evitar** herança:

|                                               | Alternativa recomendada |
|-----------------------------------------------|-------------------------|
| Relação "tem um"                              | Composição (tem_um)     |
| Subclasses não especializam claramente a base | Classes separadas       |
| Métodos herdados não se aplicam à subclasse   | Redesenhar a hierarquia |





# EXERCÍCIO PARA ENTREGA

Sua vez!



# Exercício para entrega

Classes e Objetos - Herança



## **Cenário:**

Você foi contratado para desenvolver parte do sistema de gestão de pessoas de uma universidade. O sistema deve lidar com diferentes tipos de funcionários, cada um com regras e funções específicas.



# Exercício para entrega



## Requisitos:

- Crie a classe base **Pessoa** com:
  - Atributos privados: nome, cpf e email;
  - Método: **exibir\_dados( )** que imprime todos os dados.
- Crie a classe Funcionario, que **herda** de Pessoa, com:
  - Atributo privado: salario\_base;
  - Método: **calcular\_pagamento( )** que retorna o salário base.



# Exercício para entrega

- Crie subclasses de **Funcionario**:
  - **Professor:**
    - Atributos: horas\_aula, valor\_hora;
    - O salário é calculado como:  $\text{salario\_base} + (\text{horas\_aula} * \text{valor\_hora})$ ;
    - Método sobrescrito: `calcular_pagamento( )`.
  - **Pesquisador:**
    - Atributos: bolsas, que é uma lista de valores;
    - O salário é calculado como:  $\text{salario\_base} + \text{soma}(\text{bolsas})$ ;
    - Método sobrescrito: `calcular_pagamento( )`.



# Exercício para entrega

- Crie subclasses de **Funcionario**:
  - **TecnicoAdministrativo**:
    - Recebe somente o salário base.
- Crie uma classe **Voluntario**, que **herda** apenas de **Pessoa**:
  - Não tem salário
  - Método **calcular\_pagamento( )** deve imprimir: “Voluntário – sem remuneração”



# Exercício para entrega

## □ Exemplo de uso esperado:

```
p1 = Professor("Carla", "111.222.333-44", "carla@uni.edu", 3000, 20, 50)
p2 = Pesquisador("João", "222.333.444-55", "joao@uni.edu", 2500, [500, 800])
v1 = Voluntario("Lúcia", "000.000.000-00", "lucia@uni.edu")

print("\n--- Professor ---")
p1.exibir_dados()
print(p1.calcular_pagamento())

print("\n--- Pesquisador ---")
p2.exibir_dados()
print(p2.calcular_pagamento())

print("\n--- Voluntário ---")
v1.exibir_dados()
v1.calcular_pagamento()
```



# Exercício para entrega



## **Extras (opcional):**

- Adicione um método `resumo()` que imprima o tipo de pessoa e o pagamento calculado.
- Permita cadastrar múltiplos funcionários/voluntários e exibir o total de gastos com salários.